

PHASE 7 — API Specification

Project: Prism IQ

Version: 1.0

Architecture: REST API

Protocol: HTTPS

Response Format: JSON

Backend: FastAPI

Database: PostgreSQL

Status: Design Freeze

1. API Design Principles

Every API follows these rules:

- RESTful endpoints
 - Stateless communication
 - JSON request/response
 - Predictable naming
 - Consistent error format
 - Validation before processing
 - No business logic inside routes
 - All heavy processing delegated to services
-

2. Base URL

Development

```
http://localhost:8000/api/v1
```

Production

```
https://api.prismiq.ai/api/v1
```

3. Standard Response Format

Every successful response:

```
{
  "success": true,
  "message": "Dataset uploaded successfully",
  "data": {},
  "meta": {},
  "timestamp": "2026-07-15T14:30:00Z"
}
```

Error Response

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "file",
      "message": "CSV file is required"
    }
  ],
  "timestamp": "2026-07-15T14:30:00Z"
}
```

4. API Modules

Import APIs

↓

Profile APIs

↓

Cleaning APIs

↓

EDA APIs

↓

MODULE 1 — IMPORT API

Upload CSV / Excel

POST /datasets/upload

Purpose

Upload CSV or Excel dataset.

Content-Type

multipart/form-data

Body

```
file

source_type
(csv/excel)
```

Success

```
{
  "dataset_id": "uuid",
  "dataset_name": "sales.csv",
  "rows": 15843,
  "columns": 18,
  "status": "UPLOADED"
}
```

Errors

- Invalid file
- Empty file
- Unsupported format
- File too large

Import from URL

POST /datasets/import/url

Request

```
{  
  "url": "https://example.com/table"  
}
```

Purpose

Extract HTML tables from webpages.

Response

```
{  
  "dataset_id": "uuid",  
  "tables_found": 3  
}
```

Import from API

```
POST /datasets/import/api
```

Request

```
{  
  "url": "https://api.example.com/data",  
  "headers": {  
    "Authorization": "Bearer ..."  
  }  
}
```

Response

```
{  
  "dataset_id": "uuid",  
  "records": 2453  
}
```

MODULE 2 — DATASET API

List Datasets

```
GET /datasets
```

Optional Query Parameters

```
page  
  
limit  
  
status  
  
sort  
  
search
```

Response

```
{  
  "datasets": []  
}
```

Dataset Details

```
GET /datasets/{dataset_id}
```

Returns

- Metadata
 - Status
 - Created Date
 - File Size
 - Row Count
 - Column Count
-

Delete Dataset

```
DELETE /datasets/{dataset_id}
```

Deletes

- Metadata
- Reports
- Models
- Cleaned Files

- Raw Files (configurable)

MODULE 3 — PROFILING API

Generate Profile

```
POST /profile/{dataset_id}
```

Returns

```
{
  "rows": 12543,
  "columns": 17,
  "duplicates": 15,
  "missing": 142,
  "memory": "8.2 MB",
  "quality_score": 93
}
```

Column Summary

```
GET /profile/{dataset_id}/columns
```

Returns

```
[
  {
    "column": "Age",
    "dtype": "int64",
    "missing": 3,
    "unique": 28
  }
]
```

MODULE 4 — CLEANING API

Cleaning Suggestions

```
GET /cleaning/{dataset_id}/suggestions
```

Returns

```
{
  "suggestions": [
    {
      "type": "missing_values",
      "column": "Salary",
      "recommended_action": "Median Imputation"
    }
  ]
}
```

Apply Cleaning

```
POST /cleaning/{dataset_id}
```

Request

```
{
  "remove_duplicates": true,
  "fill_missing": true,
  "standardize_categories": true,
  "handle_outliers": false
}
```

Response

```
{
  "cleaned_rows": 12495,
  "duplicates_removed": 48,
  "cleaned_file": "/uploads/cleaned/file.csv"
}
```

Download Cleaned Dataset

```
GET /cleaning/{dataset_id}/download
```

Returns cleaned CSV file.

MODULE 5 — EDA API

Generate EDA

```
POST /eda/{dataset_id}
```

Returns

Available Charts

Histogram

Bar

Pie

Scatter

Heatmap

Line

Box Plot

Correlation Matrix

```
GET /eda/{dataset_id}/correlation
```

Distribution

```
GET /eda/{dataset_id}/distribution
```

Chart Data

```
GET /eda/{dataset_id}/chart
```

Parameters

type

x

y

color

aggregation

Returns JSON optimized for Plotly/Recharts.

MODULE 6 — MACHINE LEARNING API

Detect Problem Type

```
POST /ml/{dataset_id}/detect
```

Request

```
{  
  "target": "Sales"  
}
```

Response

```
{  
  "problem": "Regression"  
}
```

Train Models

```
POST /ml/{dataset_id}/train
```

Request

```
{  
  "target": "Sales",  
  "test_size": 0.2,  
  "random_state": 42  
}
```

Response

```
{  
  "best_model": "Random Forest",  
  "accuracy": 0.96,  
}
```

```
"metrics": {}  
}
```

Prediction

```
POST /ml/{dataset_id}/predict
```

Request

```
{  
  "input": {  
    "Age": 25,  
    "Income": 45000  
  }  
}
```

Response

```
{  
  "prediction": 51200  
}
```

Model Comparison

```
GET /ml/{dataset_id}/models
```

Returns all trained models and evaluation metrics.

MODULE 7 — INSIGHTS API

Generate Recommendations

```
POST /insights/{dataset_id}
```

Returns

```
{  
  "recommendations": [  
    {  
      "priority": "HIGH",  
      "title": "Increase Electronics Inventory",  
      "confidence": 94  
    }  
  ]  
}
```

```
}  
]  
}
```

Executive Summary

```
GET /insights/{dataset_id}/summary
```

Returns AI-generated (rule-based MVP) business summary.

MODULE 8 — REPORT API

Generate Report

```
POST /reports/{dataset_id}
```

Request

```
{  
  "type": "executive"  
}
```

Response

```
{  
  "report_id": "uuid",  
  "status": "GENERATED"  
}
```

Download Report

```
GET /reports/{report_id}/download
```

Returns PDF.

Report Preview

```
GET /reports/{report_id}/preview
```

Returns structured preview data for frontend.

MODULE 9 — SYSTEM API

Health Check

```
GET /health
```

Response

```
{
  "status": "healthy",
  "database": "connected",
  "version": "1.0.0"
}
```

5. HTTP Status Codes

Code	Meaning
200	Success
201	Resource Created
202	Accepted (Future async jobs)
204	Deleted Successfully
400	Bad Request

401	Unauthori zed (Future)
403	Forbidden
404	Not Found
409	Conflict
413	Payload Too Large
415	Unsupport ed Media Type
422	Validation Error
429	Too Many Requests (Future)
500	Internal Server Error

6. Validation Rules

Upload

- CSV / XLSX only (MVP)
- Maximum size: **100 MB**
- UTF-8 encoding preferred
- First row must contain headers
- Empty datasets rejected

URL Import

- Valid HTTP/HTTPS URL
- At least one HTML table required

API Import

- JSON response required
- Array of objects expected

ML

- Target column mandatory
- Minimum 50 rows recommended
- Minimum 2 feature columns required

7. Pagination Standard

```
GET /datasets?page=1&limit=20
```

Response metadata:

```
{
  "page": 1,
  "limit": 20,
  "total": 145,
  "pages": 8
}
```

8. Versioning Strategy

Current:

```
/api/v1/
```

Future:

```
/api/v2/
```

No breaking changes inside a version.

9. OpenAPI Tags

```
Datasets
Import
Profiling
```

Cleaning
EDA
Machine Learning
Insights
Reports
System

These tags organize Swagger UI automatically.

10. Security (MVP & Future)

MVP

- No authentication required for local development.
- Input validation on all endpoints.
- Safe file handling.
- File type and size validation.

Future

- JWT Authentication
 - User accounts
 - Project ownership
 - Rate limiting
 - Role-based access
-

11. API Flow Summary

```
graph TD; A[Upload Dataset] --> B[Generate Profile]; B --> C[Cleaning Suggestions]; C --> D[Apply Cleaning]; D --> E[Generate EDA];
```

Upload Dataset
▼
Generate Profile
▼
Cleaning Suggestions
▼
Apply Cleaning
▼
Generate EDA

Train ML Models

✓ API Design Freeze

By freezing this specification now:

- Backend implementation has a clear contract.
- Frontend can integrate without waiting for backend internals.
- Swagger/OpenAPI documentation will naturally mirror this structure.
- Future authentication and asynchronous processing can be added without changing endpoint organization.

🔥 One architectural improvement I recommend

Since Prism IQ is **pipeline-based**, I'd slightly adjust the endpoint naming to reflect that.

Instead of:

```
POST /profile/{dataset_id}
POST /cleaning/{dataset_id}
POST /eda/{dataset_id}
```

I'd use:

```
POST /datasets/{dataset_id}/profile
POST /datasets/{dataset_id}/clean
POST /datasets/{dataset_id}/eda
POST /datasets/{dataset_id}/train
POST /datasets/{dataset_id}/insights
POST /datasets/{dataset_id}/report
```

This keeps **datasets** as the **root resource**, making the API more RESTful and easier to understand. It also scales better if you later introduce projects, users, or workspaces. I recommend adopting this structure before any code is written, because changing endpoint paths later affects both frontend and backend.